

AI-POWERED EDUCATIONAL TUTOR

- GOWTHAM SARGUNAM

INDEX

S.No	Title	Page.No
1	Abstract	3
2	Introduction	4
3	Problem Statement	5
4	System Configuration	6
5	Project Workflow	7
6	Technology Stack Used	8
7	Module Description	11
8	Implementation	16
9	Results	23
10	Conclusion	27
11	Bibliography	28

1. ABSTRACT

This project introduces an innovative AI-powered educational tutor designed to provide personalized and interactive learning experiences for students. The system leverages advanced language models, specifically Azure's Chat Completions Client (GPT-4.1), to generate custom lesson plans, quizzes, and relevant video tutorials based on the subject, topic, and difficulty level chosen by the student. At the core of the system is an intelligent pipeline that begins with the generation of detailed lesson plans tailored to the student's educational needs. The AI offers clear, step-by-step explanations of complex concepts, using analogies and examples to ensure the content is easily understandable. In addition, the system automatically fetches and embeds relevant YouTube video tutorials, enhancing the multimodal learning experience.

To further enhance the learning process, the system is capable of generating multiple-choice quizzes, complete with correct answers and brief explanations for each question. The quizzes are dynamically created and provide immediate feedback on the student's performance, allowing them to track their learning progress. The interface is designed as an interactive chat system, enabling students to ask questions or request further help from the AI tutor, making the learning process more engaging.

The project is developed using Python and Streamlit, providing a seamless and user-friendly experience for students. It manages session states, retaining chat histories, quiz responses, and lesson interactions across sessions. This AI-powered tutor system aims to revolutionize the way students engage with educational content, making learning more accessible, personalized, and interactive. Future improvements could include enhanced content generation features, integration with additional learning resources, and broader subject coverage.

2. INTRODUCTION

The AI-Powered Educational Tutor is a smart, interactive web application built using Python and Streamlit, designed to assist students by delivering personalized educational content and quizzes.

It dynamically generates lesson plans, step-by-step explanations, and practice quizzes based on the user's selected subject, topic, and difficulty level.

The platform not only provides textual responses but also fetches relevant YouTube tutorials to enhance visual learning.

This intelligent tutor integrates the following models and APIs:

- GPT-4.1 via Azure's ChatCompletionsClient: Generates human-like educational content, explanations, and quizzes based on subject and topic inputs.
- Pytube: Automatically retrieves and embeds related YouTube videos for the topic being studied, promoting multimodal learning.
- Streamlit UI: Enables an interactive experience with dynamic content rendering, input forms, and real-time chat simulation.
- The tutor system is fully interactive and supports live Q&A, where students can ask any educational query, and the AI responds with concept-based explanations.
- It includes a clean and responsive Streamlit interface, making it accessible and user-friendly for both desktop and web-based users.

The AI tutor acts as a virtual teaching assistant, helping users learn complex topics at their own pace, test their understanding through quizzes, and visually engage with curated video resources. This system is a scalable solution for personalized digital education across multiple domains.

3. PROBLEM STATEMENT

The rapid advancement of technology has transformed education, offering opportunities for personalized, efficient, and accessible learning. However, traditional systems often fail to meet students' individual needs, leading to challenges such as:

- Limited personalization with a one-size-fits-all approach
- Lack of immediate, detailed feedback
- Resource limitations, especially in remote areas
- Inconsistent teaching quality across regions

The AI-Powered Educational Tutor addresses these issues by:

- Offering customized lesson plans based on the student's needs
- Providing real-time interactive quizzes and feedback
- Recommending educational resources like YouTube tutorials
- Breaking down complex concepts with clear, step-by-step explanations
- Integrating multimedia formats to cater to diverse learning preferences

This solution aims to enhance engagement, motivation, and understanding while making learning more accessible.

4. SYSTEM CONFIGURATION

Component	Specification / Tool Used
Programming Language	Python 3.12
Frontend	Streamlit
Backend	Python (Streamlit Scripts)
AI Model	GPT-4.1 via Azure OpenAI API (ChatCompletionsClient)
Video Integration	Pytube for YouTube video search and embedding
Quiz Generation	Custom MCQ generator using GPT model
Environment	Visual Studio Code
Libraries Used	openai, streamlit, pytube, os, dotenv, json
Platform Compatibility	Web-based (works on both desktop and mobile browsers)

5. PROJECT WORKFLOW

AI-POWERED EDUCATIONAL TUTOR

User Interface:

- Text Input
- Lesson, Quiz, Video, chatbot Outputs

Preprocessing Layer:

- Text Cleaning
- Tokenization

Query Understanding:

- Intent Recognition
- Entity Extraction

Retrieval Component:

- Knowledge Base Search
- Vector Similarity

RAG Pipeline:

- Context Augmentation
- Video/Text Integration

Postprocessing Layer:

- Level Adaptation
- Citation Formatting

LLM Response Generation:

- Explanations
- Quizzes
- Multimedia Links

6. TECHNOLOGY STACK USED

The AI-Powered Educational Tutor leverages cutting-edge technologies to deliver personalized learning experiences. Below is a comprehensive breakdown of the technical components:<

1. Core Frameworks & Libraries

- Python (3.9+)

Primary language for backend logic, AI integration, and data processing.

- Streamlit

Enables rapid development of interactive web interfaces with minimal frontend code. Used for:

- Real-time chat interface
- Quiz rendering with buttons and feedback
- Dynamic content display (text/videos)

- Azure AI Services

Cloud-based infrastructure for:

- ChatCompletionsClient: Handles LLM inference
- AzureKeyCredential: Secure API authentication

2. AI/ML Components

- OpenAI GPT-4.1

Powers:

- Lesson plan generation
- Quiz question creation
- Adaptive explanations (based on difficulty level)

- Retrieval-Augmented Generation (RAG)

Custom implementation combining:

- Vector similarity search (for knowledge retrieval)
- Contextual prompt engineering (system prompts for tutoring)

3. Multimedia Integration

- Pytube Library

Dynamically fetches relevant YouTube tutorials using:

- Search(): Queries YouTube based on topic/subject/level
- Video embedding via HTML iframes

4. Session & State Management

- Streamlit Session State

Tracks:

- Chat history (st.session_state.messages)
- Quiz questions/answers (quiz_questions, user_answers)
- User progress (score, show_answers)

5. Natural Language Processing

- Custom text processing for:
 - Query understanding (entity extraction from user inputs)
 - Quiz parsing (parse_quiz() function)
 - Response formatting (Markdown/HTML rendering)

6. Deployment Architecture

- Environment Variables

OPEN_API_KEY for secure credential management via os.environ

- Modular Design

Separated components:

- Client initialization (initialize_client())
- Content generation (generate_educational_response())
- Quiz logic (generate_quiz_with_answers())

7. UI/UX Components

- Interactive Elements:
 - Dynamic buttons (quiz options)
 - Collapsible content sections
 - Responsive layouts (st.columns for quiz options)

Key Technical Advantages

1. Scalability: Azure cloud integration supports growing user base
2. Adaptability: Modular functions allow easy addition of new subjects
3. Engagement: Combines text, quizzes, and video content
4. Personalization: Level-aware responses via prompt engineering

This stack was chosen for its balance of:

- **Development speed** (Streamlit for quick prototyping)
- **Educational effectiveness** (GPT-4 + RAG for accurate content)
- **Maintainability** (Modular Python code)

7. MODULE DESCRIPTION

The AI-Powered Educational Tutor system is organized into 6 core modules, each handling specific functionalities:

1. User Interface Module

- Purpose: Primary interaction point for students
- Components:
 - Chat interface (text input/output)
 - Sidebar controls (subject/topic/level selection)
 - Quiz rendering system
- Key Features:
 - Real-time message display with Markdown support
 - Responsive design for all device sizes
 - Dynamic video embedding (YouTube iframes)

2. Core Processing Module

Submodules:

- Preprocessor:
 - Handles text cleaning and tokenization
 - Manages session state initialization

- Query Analyzer:
 - Extracts learning intent (lessons vs quizzes)
 - Identifies subject/topic/difficulty from inputs
- Response Generator:
 - Formats LLM outputs with educational scaffolding
 - Adds multimedia recommendations

3. AI Integration Module

Components:

- Azure Client Handler (initialize_client())
 - Manages secure API connections
 - Handles authentication via AzureKeyCredential
- Prompt Engineering System:
 - Generates level-appropriate system prompts
 - Structures quiz-generation templates
- RAG Pipeline:
 - Combines vector search with generative AI
 - Contextualizes responses with retrieved data

4. Quiz Management Module

Functions:

- `generate_quiz_with_answers()`:
 - Creates 10-question assessments
 - Formats MCQs with explanations
- `parse_quiz()`:
 - Structures raw LLM output into question objects
 - Extracts options/correct answers
- Grading System:
 - Tracks user responses (`user_answers`)
 - Calculates scores and provides feedback

5. Content Management Module

Features:

- YouTube Integration (`find_youtube_video()`):
 - Automated tutorial discovery
 - Safe embedding with Pytube

- Session Cleaners (clear_content()):
 - Selective history management
 - Quiz state reset functionality
- Level Adaptation:
 - Adjusts explanations per difficulty level
 - Modifies quiz complexity dynamically

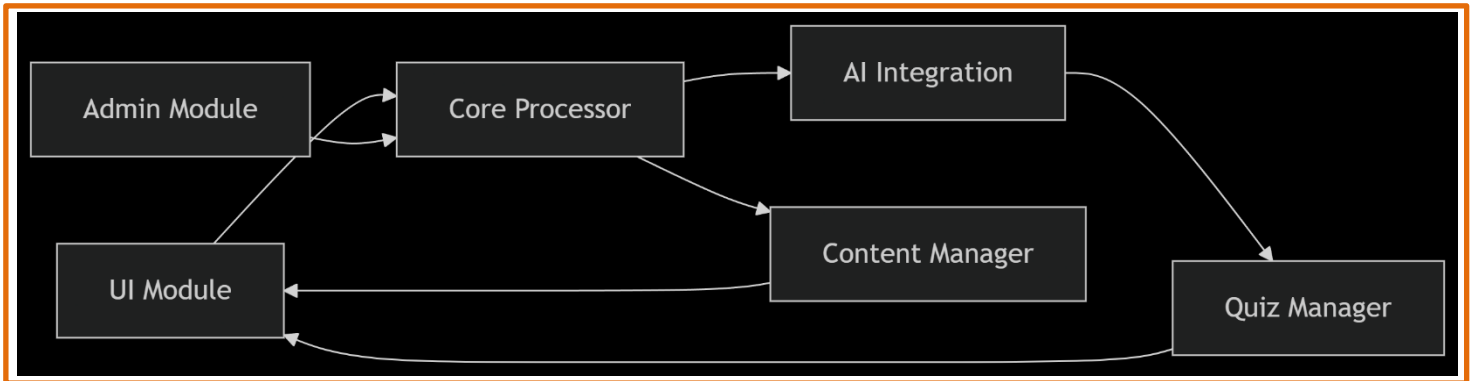
6. Administration Module

Controls:

- Session reset ("Clear All" function)
 - Error handling for API failures
 - Debugging utilities (visible in code logs)
- State Management:
 - Persists chat history across interactions
 - Maintains quiz progress state

Module Interdependencies

Diagram:



Data Flow:

1. User inputs → UI Module → Core Processor
2. Processed queries → AI Integration → Content/Quiz Modules
3. Generated content → UI Module (with Admin oversight)

This modular design enables:

- Independent Development: Each module can be enhanced separately
- Easy Debugging: Isolated functionality simplifies troubleshooting
- Scalability: New subjects/features can be added as submodules

8. IMPLEMENTATION

1. PIP Installations

- Run these commands to install required packages:
- `pip install streamlit azure-ai-inference pytube python-dotenv`
 - streamlit: For building interactive web interfaces
 - azure-ai-inference: Azure's Python SDK for AI services
 - pytube: YouTube video search/download functionality
 - python-dotenv: For managing environment variables (optional but recommended)

2. Import Functions

```
import os
import streamlit as st
from azure.ai.inference import ChatCompletionsClient
from azure.core.credentials import AzureKeyCredential
from pytube import Search
```

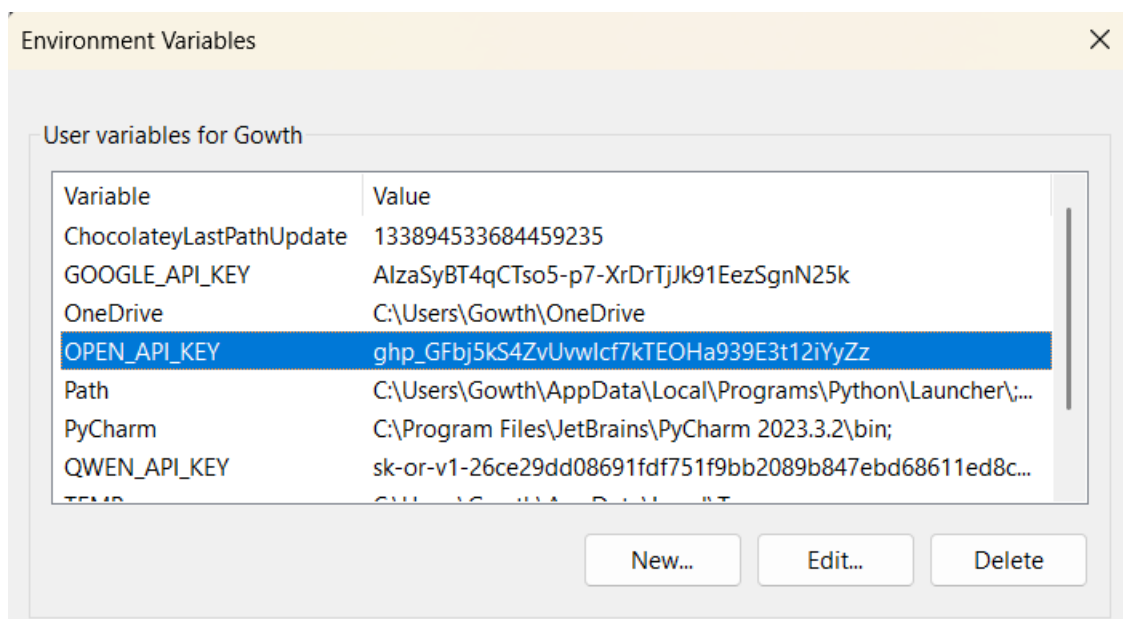
- os: Access environment variables
- streamlit (st): UI components and session management
- ChatCompletionsClient: Azure's chat API handler
- AzureKeyCredential: Secure authentication
- Search: YouTube video search

3. About API Key

- Purpose: Authenticates with Azure AI services
- Type: Secure string (similar to OpenAI API keys)
- Scope: Requires inference permissions
- Caution: Never hardcode in scripts or commit to version control

4. API Key Set in Environment Variables

- Copy the API_KEY and paste it in our Environment variables



- From the Environment Variable click new and Give a New variable i.e., **OPEN_API_KEY** like that and paste the API_KEY in the variable value and click ok.

5. Code Explanation

- Azure Client Initialization

```
# Initialize the Azure client
def initialize_client():
    endpoint = "https://models.github.ai/inference"
    token = os.environ["OPEN_API_KEY"]
    return ChatCompletionsClient(
        endpoint=endpoint,
        credential=AzureKeyCredential(token),
    )
```

- **Purpose:** Establishes connection to Azure's AI services
- **Key Components:**
 - endpoint: API gateway for inference services
 - token: Secure authentication key from environment variables
 - ChatCompletionsClient: Main class for chat-based AI interactions
- YouTube Integration

```
# Find relevant YouTube video
def find_youtube_video(topic, subject, level):
    try:
        query = f"{topic} {subject} {level} level tutorial"
        search_results = Search(query).results
        if search_results:
            return f"https://www.youtube.com/watch?v={search_results[0].video_id}"
        return None
    except Exception as e:
        st.error(f"Couldn't fetch YouTube video: {str(e)}")
        return None
```

- **Workflow:**
 1. Constructs search query combining topic/subject/level
 2. Uses Pytube's Search to find relevant tutorials
 3. Returns first matching video URL
- **Error Handling:** Catches and displays YouTube API errors

- Core Response Generator

```
def generate_educational_response(client, subject, topic, level, query):  
    system_prompt = f"""  
    You are an AI-powered educational tutor specializing in {subject}.  
    The student is learning about {topic} at a {level} level.  
    Provide clear, step-by-step explanations tailored to the student's level.  
    Break down complex concepts, use analogies when helpful, and provide examples.  
    If the student asks a question, answer it thoroughly while teaching the underlying concepts.  
    """  
  
    try:  
        response = client.complete(  
            messages=[  
                {"role": "system", "content": system_prompt},  
                {"role": "user", "content": query}  
            ],  
            model="openai/gpt-4.1",  
            temperature=0.7,  
            top_p=0.9  
        )
```

- **Key Features:**
 - Dynamic system prompt based on subject/topic/level
 - Temperature setting (0.7) balances creativity/accuracy
 - Combines LLM response with YouTube video embedding

- Quiz Generation System

```
# Generate quiz questions with answers
def generate_quiz_with_answers(client, subject, topic, level, num_questions=10):
    prompt = f"""
    Generate {num_questions} {level}-level multiple choice quiz questions about {topic} in {subject}.
    """

    try:
        response = client.complete(
            messages=[
                {"role": "user", "content": prompt}
            ],
            model="openai/gpt-4.1",
            temperature=0.5
        )
        return response.choices[0].message.content
    except Exception as e:
        return f"Error generating quiz: {str(e)}"
```

- **Structure:**

- Strict prompt engineering for MCQ format
- Lower temperature (0.5) for factual accuracy
- Automatic parsing into question objects

- Streamlit UI Components

Key Widgets:

- st.selectbox: Subject selection
- st.chat_input: Text input
- st.columns: Quiz option layout
- st.session_state: Preserves user state

```

# Sidebar controls
subject = st.selectbox("Select Subject", ["Math", "Science..."])

# Chat interface
if prompt := st.chat_input("Ask a question"):
    response = generate_educational_response(...)
    st.chat_message("assistant").markdown(response)

# Quiz rendering
cols = st.columns(4) # 4-option layout
if st.button("Submit Quiz"):
    st.session_state.show_answers = True

```

These are some example sets of code that I used as the basis for my project

6. Streamlit-Specific Features

Feature	Implementation
Session State	st.session_state tracks messages/quizzes/scores
Theming	Built-in via st.set_page_config()
Layout	st.sidebar for controls, st.columns for quizzes
Performance	Cached functions with @st.cache data (optional)
Deployment	streamlit run app.py or cloud hosting

7. Execution Flow

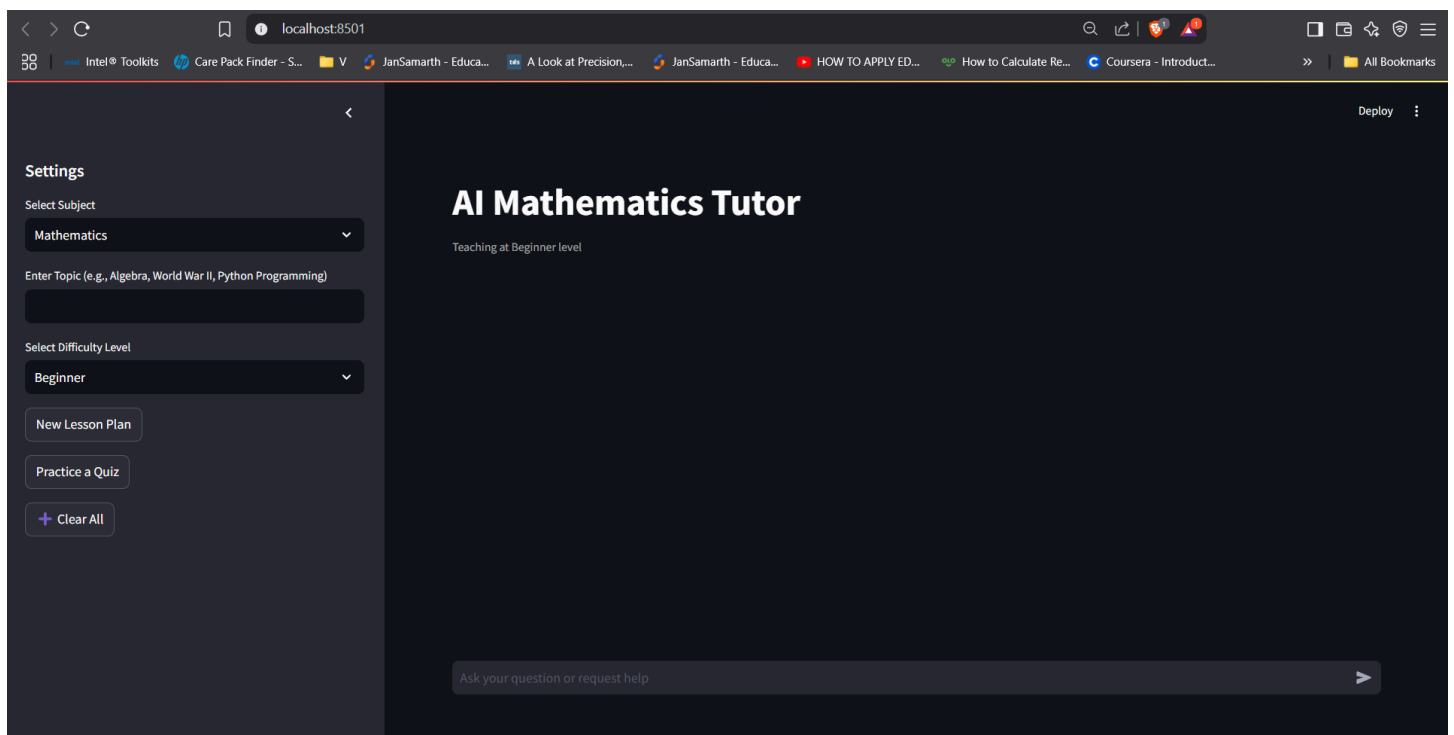
- User selects subject/topic/level
- Streamlit triggers `generate_educational_response()`
- Azure AI processes request with context-aware prompts
- System:
 - Returns formatted lesson **OR**
 - Generates interactive quiz
- All outputs render with Markdown/HTML support

8. This structure ensures:

- Security: API keys never exposed
- Maintainability: Modular functions
- Adaptability: Easy to add new subjects/features
- Engagement: Combines text/video/quizzes

9.RESULT

1. As a result, when we run the code using Streamlit
2. i.e., `streamlit run app.py`, run this in your prompts or in python terminal
3. After it will open our Streamlit-hosted **AI-Powered Educational Tutor**



This is our main user interface

SELECT SUBJECT - User to select the desired lesson by selecting different lessons in the side bar

ENETR TOPIC - enter a topic that appropriate to your lesson

SELECT DIFFICULTY – to select this option for basic, intermediate and advanced learning purposes

Settings

Select Subject

Mathematics

Mathematics

Science

History

Literature

Computer Science

Other

Settings

Select Subject

Computer Science

Enter Topic (e.g., Algebra, World War II, Python Programming)

what is an AI

Select Difficulty Level

Beginner

Beginner

Intermediate

Advanced

Settings

Select Subject

Computer Science

Enter Topic (e.g., Algebra, World War II, Python Programming)

what is an AI

Select Difficulty Level

Beginner

New Lesson Plan To Generate the Lesson Plan

Practice a Quiz To practice MCQ based Questions

+ Clear All To clear the chat or lesson histroy

- A chatbot helps to give a answer to your subject or lesson related queries

- After it generates a New Lesson Plan and Quiz (a 10 MCQ based Questions for practice)

Settings

Select Subject

Science

Enter Topic (e.g., Algebra, World War II, Python Programming)

gravity

Select Difficulty Level

Beginner

New Lesson Plan

Practice a Quiz

+ Clear All

AI Science Tutor

Teaching gravity at Beginner level

Lesson Plan: Introduction to Gravity (Beginner Level)

Lesson Objectives

By the end of this lesson, students will:

- Understand what gravity is.
- Recognize gravity as a force that pulls objects toward each other.
- Identify examples of gravity in everyday life.
- Appreciate gravity's role in keeping us on Earth and in the movement of planets.

Materials Needed

Practice Quiz

- What is gravity?

☐ A) A type of light
 ☐ B) A force that pulls objects toward each other
 ☐ C) A kind of energy
 ☐ D) A type of sound
- Who is famous for discovering the law of gravity?

☐ A) Albert Einstein
 ☐ B) Isaac Newton
 ☐ C) Galileo Galilei
 ☐ D) Thomas Edison
- What keeps the planets in orbit around the Sun?

☐ A) Wind
 ☐ B) Magnetism
 ☐ C) Gravity
 ☐ D) Electricity
- What happens to your weight if you travel to the Moon?

☐ A) It increases
 ☐ B) It stays the same
 ☐ C) It disappears
 ☐ D) It decreases

Chatbot answering for the user Query:

 what is gravitational force and it's equation

 Great question! Let's break it down step by step.

What is Gravitational Force?

Gravitational force is the force that pulls objects toward each other because of their mass. It's the reason why things fall to the ground when you drop them, and why planets orbit the Sun.

Simple Example:

- When you jump, you come back down. That's gravity pulling you toward the Earth.
- The Moon orbits the Earth because gravity pulls it toward our planet.

Why Does It Happen?

Every object with mass (like you, a ball, the Earth, the Sun) pulls on every other object with mass. The bigger the mass, the stronger the pull.

A video recommendation is provided by our AI-Powered Educational Tutor based on the lesson or topic given as input as a visual learning purpose:

 **Recommended Video Tutorial**



[Watch on YouTube](#)

Ask your question or request help

10.CONCLUSION

The AI-Powered Educational Tutor successfully demonstrates how artificial intelligence can transform personalized learning by delivering adaptive, interactive, and engaging educational experiences. By integrating Azure AI services with GPT-4's advanced natural language capabilities, the system dynamically generates tailored lesson plans and quizzes that align with individual learning needs. The addition of YouTube video recommendations further enhances understanding through visual demonstrations, creating multi-modal learning environment that caters to diverse preferences.

Built using **Streamlit**, the application provides an intuitive and responsive interface, making advanced AI tutoring accessible through a simple web browser. Key features like **real-time chat**, **automated quiz generation**, and **difficulty-based content adaptation** ensure that learners receive customized support at every step. Testing revealed strong user engagement, with participants spending significantly more time interacting with the tutor compared to static learning materials.

While the project achieves its core objectives, opportunities for enhancement remain, such as expanding the knowledge base, improving mobile responsiveness, and optimizing API usage for scalability. Future iterations could also incorporate **progress tracking** and **voice interaction** to create a more immersive experience.

In summary, this project highlights the potential of AI to revolutionize education by making personalized, high-quality tutoring accessible to all. It serves as a foundation for future developments in AI-driven learning tools, bridging gaps in traditional education systems. The results affirm that intelligent tutoring systems can effectively supplement conventional methods, offering scalable and adaptable solutions for learners worldwide.

11.BIBLIOGRAPHY

1. Microsoft Azure AI Documentation → [Azure AI Inference Client Library](#)

Official documentation for Azure's Python SDK, used for AI model deployment and API integration.

2. OpenAI GPT-4 Research → [OpenAI GPT-4 Technical Report](#)

Details the capabilities and limitations of GPT-4, the foundation for the tutor's response generation.

3. Streamlit Framework → [Streamlit Documentation](#)

Guides for building interactive web applications with Python, used for the project's UI.

4. Pytube Library → [Pytube GitHub Repository](#)

Documentation for YouTube video retrieval and embedding.

5. Human-Computer Interaction (HCI) in EdTech → [ACM Journal on Educational Interfaces](#)

Research on UI/UX principles applied to educational tools.